



TRANSITION TO C++

By: Gretchen Y. Bonilla

Department of Computer Science and Engineering

University of Puerto Rico, Mayaguez

OBJECTIVES



Review variables, data types, expressions, and statements.



Intro to C++



Intro to VSCode.

INTRO TO PROGRAMMING

Let's see what we are getting ourselves into

WHY PROGRAM?



COMPUTER

They follow instructions.
Programmable.

PROGRAM/SOFTWARE

Gives instructions.

PROGRAMMER

Designs, implements and tests programs.

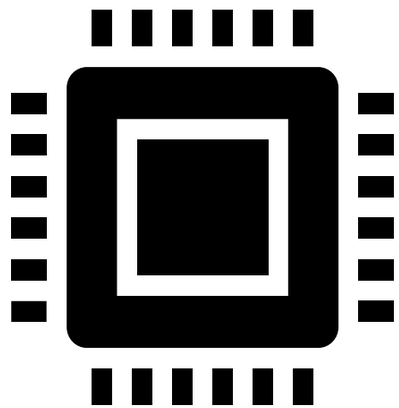
HARDWARE AND SOFTWARE

We might like one more than the other, but both are equally important.

HARDWARE

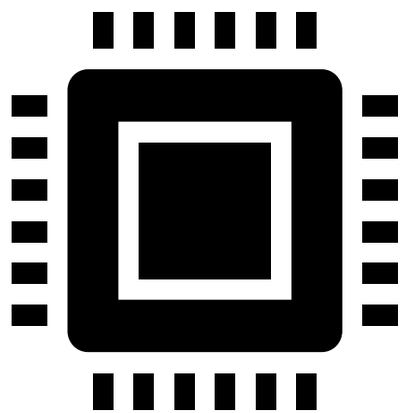
Physical components of the computer

- Central Processing Unit (CPU)
  - Like brain of the computer.
  - Fetches instructions and coordinates its execution.
- Memory (RAM):
  - Fast and volatile memory.
  - Stores data and instructions.
- Secondary Memory:
  - Slower but can hold data long term.



HARDWARE

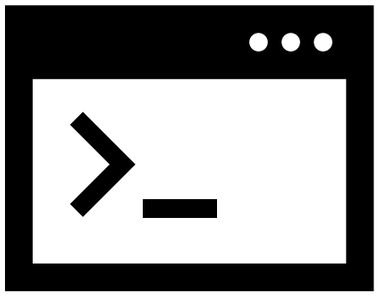
- Input Devices:
  - Sends information to the computer
  - e.g., keyboards, cameras, microphones, etc.
- Output devices:
  - Sends information out of the computer
  - e.g., printer, monitors, USB drive, etc.



SOFTWARE

Most frequent software:

- Operating Systems:
  - Controls internal operations of computer.
  - Connects hardware with software.
  - Example: Windows, MacOS, Ubuntu, etc.
- Utility programs:
  - Supports internal operations.
  - Antivirus, file compression, etc.
- Software development tools:
  - Used to create software.
  - Compilers and integrated development environments (IDEs) such as Eclipse, PyCharm, IntelliJ, Visual Code Studio.



PROGRAMS

- Computer programs: Set of instructions that tell the computer how to solve a problem.
- An algorithms is a series of *sequential* steps to solve a problem.
  - Like a recipe
  - There can be <u>more than one</u> algorithm to solve one problem.
  - For example, there can be more than one route to the same location.



PROGRAMS

A programming language is a special language used to write programs.

Types of languages:

- Low level: Closer to the hardware, not easy to understand.
  - Machine language and Assembly
  - More efficient since close to hardware
- High level: More like human language but needs to be translated for computer to understand.
  - C, C++, Java, Python, JavaScript, etc.
  - Slower since code need to be translated to machine language

LOW LEVEL VS HIGH LEVEL

ASSEMBLY

```
section .data
  hello:     db 'Hello world!',10    ; 'Hello world!' plus a linefeed character
  helloLen:  equ $-hello             ; Length of the 'Hello world!' string

section .text
  global _start

_start:
  mov eax,4            ; The system call for write (sys_write)
  mov ebx,1            ; File descriptor 1 - standard output
  mov ecx,hello        ; Put the offset of hello in ecx
  mov edx,helloLen     ; helloLen is a constant, so we don't need to say
                       ;  mov edx,[helloLen] to get it's actual value
  int 80h              ; Call the kernel
  mov eax,1            ; The system call for exit (sys_exit)
  mov ebx,0            ; Exit with return code of 0 (no error)
  int 80h;
```

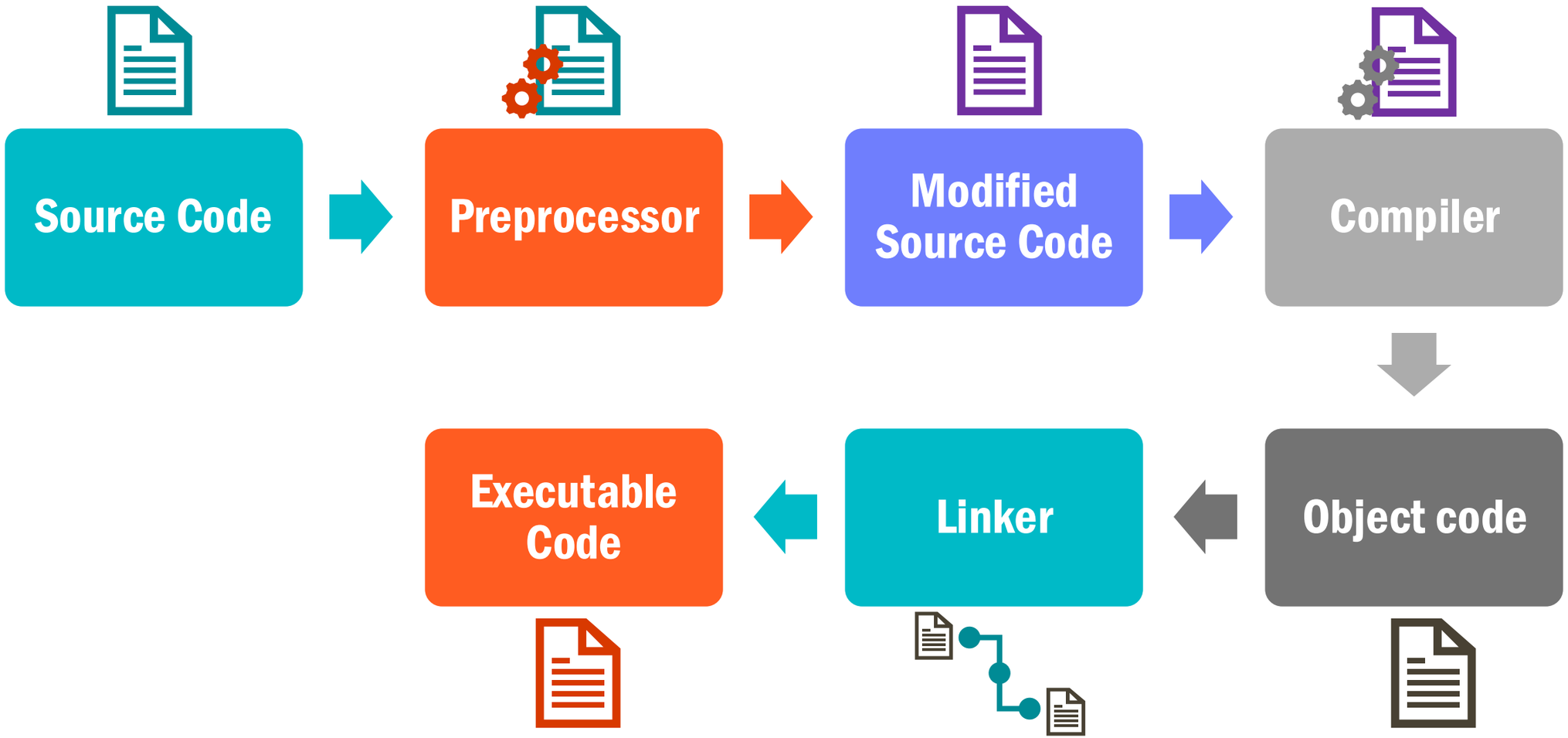
Code from:
https://onecompiler.com/assembly/3uccjrhsa

C++

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello, World!" << endl;
    return 0;
}
```

PROGRAMS


Source Code
Preprocessor
Modified Source Code
Compiler
Object code
Linker
Executable Code

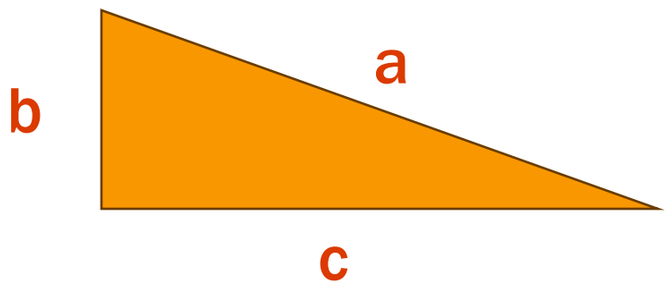
THE BASICS

We can achieve the same thing; it's just said differently.

EXAMPLE 1: TRIANGLES

Let's implement a program that does the following:

- Calculates the area of a triangle
- We get the length of all three sides from the console
- We use Heron's formula for the area.
- We print the area.



$$s = \frac{a + b + c}{2}$$

$$A = \sqrt{s(s - a)(s - b)(s - c)}$$

EXAMPLE 1: TRIANGLES IN PYTHON

```
import math

sideA = int(input("What is the length of A?"))
sideB = int(input("What is the length of B?"))
sideC = int(input("What is the length of C?"))

S = (sideA + sideB + sideC)/2

Area = math.sqrt(S *(S-sideA)*(S-sideB)*(S-sideC))

print(Area)
```

EXAMPLE 1: TRIANGLES IN C

```c
#include<stdio.h>
#include<math.h>

int main() {
  int sideA, sideB, sideC;
  printf("What is side A? ");
  scanf("%d", &sideA);
  printf("What is side B? ");
  scanf("%d", &sideB);
  printf("What is side C? ");
  scanf("%d", &sideC);
  float s = (sideA + sideB + sideC)/2;
  float area = sqrt(s*(s-sideA)*(s-sideB) * (s-sideC));
  printf("Area is: %f", area);
  return 0;
}
```

EXAMPLE 1: PYTHON VS C

PYTHON

```python
import math

sideA = int(input("What is the length of A?"))
sideB = int(input("What is the length of B?"))
sideC = int(input("What is the length of C?"))

S = (sideA + sideB + sideC)/2

Area = math.sqrt(S *(S-sideA)*(S-sideB)*(S-sideC))

print(Area)
```

C

```c
#include<stdio.h>
#include<math.h>

int main() {
  int sideA, sideB, sideC;
  printf("What is side A? ");
  scanf("%d", &sideA);
  printf("What is side B? ");
  scanf("%d", &sideB);
  printf("What is side C? ");
  scanf("%d", &sideC);
  float s = (sideA + sideB + sideC)/2;
  float area = sqrt(s*(s-sideA)*(s-sideB) * (s-sideC));
  printf("Area is: %f", area);
  return 0;
}
```

Both do the same actions and achieve the same result, but the way it's written changes.

EXAMPLE 1: PYTHON VS C

PYTHON

```python
import math

sideA = int(input("What is the length of A?"))
sideB = int(input("What is the length of B?"))
sideC = int(input("What is the length of C?"))

S = (sideA + sideB + sideC)/2

Area = math.sqrt(S *(S-sideA)*(S-sideB)*(S-sideC))

print(Area)
```

C

```c
#include<stdio.h>
#include<math.h>

int main() {
  int sideA, sideB, sideC;
  printf("What is side A? ");
  scanf("%d", &sideA);
  printf("What is side B? ");
  scanf("%d", &sideB);
  printf("What is side C? ");
  scanf("%d", &sideC);
  float s = (sideA + sideB + sideC)/2;
  float area = sqrt(s*(s-sideA)*(s-sideB) * (s-sideC));
  printf("Area is: %f", area);
  return 0;
}
```

In both cases we need imports so we can use additional functions to achieve our goals. In C, we use header files.

EXAMPLE 1: PYTHON VS C

PYTHON

```python
import math

sideA = int(input("What is the length of A?"))
sideB = int(input("What is the length of B?"))
sideC = int(input("What is the length of C?"))

S = (sideA + sideB + sideC)/2

Area = math.sqrt(S *(S-sideA)*(S-sideB)*(S-sideC))

print(Area)
```

C

```c
#include<stdio.h>
#include<math.h>

int main() {
  int sideA, sideB, sideC;
  printf("What is side A? ");
  scanf("%d", &sideA);
  printf("What is side B? ");
  scanf("%d", &sideB);
  printf("What is side C? ");
  scanf("%d", &sideC);
  float s = (sideA + sideB + sideC)/2;
  float area = sqrt(s*(s-sideA)*(s-sideB) * (s-sideC));
  printf("Area is: %f", area);
  return 0;
}
```

Both print to the console and receive the user's input. But C's way of doing this requires more steps.

EXAMPLE 1: TRIANGLES

PYTHON

```python
import math

sideA = int(input("What is the length of A?"))
sideB = int(input("What is the length of B?"))
sideC = int(input("What is the length of C?"))

S = (sideA + sideB + sideC)/2

Area = math.sqrt(S *(S-sideA)*(S-sideB)*(S-sideC))

print(Area)
```

C

```c
#include<stdio.h>
#include<math.h>

int main() {
  int sideA, sideB, sideC;
  printf("What is side A? ");
  scanf("%d", &sideA);
  printf("What is side B? ");
  scanf("%d", &sideB);
  printf("What is side C? ");
  scanf("%d", &sideC);
  float s = (sideA + sideB + sideC)/2;
  float area = sqrt(s*(s-sideA)*(s-sideB) * (s-sideC));
  printf("Area is: %f", area);
  return 0;
}
```

Python can pass the input to the variable at declaration. In C, we must first create the variables.

EXAMPLE 1: PYTHON VS C

PYTHON

```python
import math

sideA = int(input("What is the length of A?"))
sideB = int(input("What is the length of B?"))
sideC = int(input("What is the length of C?"))

S = (sideA + sideB + sideC)/2

Area = math.sqrt(S *(S-sideA)*(S-sideB)*(S-sideC))

print(Area)
```

C

```c
#include<stdio.h>
#include<math.h>

int main() {
  int sideA, sideB, sideC;
  printf("What is side A? ");
  scanf("%d", &sideA);
  printf("What is side B? ");
  scanf("%d", &sideB);
  printf("What is side C? ");
  scanf("%d", &sideC);
  float s = (sideA + sideB + sideC)/2;
  float area = sqrt(s*(s-sideA)*(s-sideB) * (s-sideC));
  printf("Area is: %f", area);
  return 0;
}
```

In C we must indicate the data type if the variable being created, whereas in Python that's not necessary.

EXAMPLE 1: PYTHON VS C

PYTHON

```python
import math

sideA = int(input("What is the length of A?"))
sideB = int(input("What is the length of B?"))
sideC = int(input("What is the length of C?"))

S = (sideA + sideB + sideC)/2

Area = math.sqrt(S *(S-sideA)*(S-sideB)*(S-sideC))

print(Area)
```

C

```c
#include<stdio.h>
#include<math.h>

int main() {
  int sideA, sideB, sideC;
  printf("What is side A? ");
  scanf("%d", &sideA);
  printf("What is side B? ");
  scanf("%d", &sideB);
  printf("What is side C? ");
  scanf("%d", &sideC);
  float s = (sideA + sideB + sideC)/2;
  float area = sqrt(s*(s-sideA)*(s-sideB) * (s-sideC));
  printf("Area is: %f", area);
  return 0;
}
```

To execute code in Python we can write directly in the file. In C, the code that needs to be inside a main() function.

EXAMPLE 1: TRIANGLES IN C++

```
#include<iostream>
#include<math.h>
using namespace std;

int main() {
  int sideA, sideB, sideC;

  cout << "What is side A?";
  cin >> sideA;
  cout<< "What is side B?";
  cin >> sideB;
  cout<< "What is side C?";
  cin >> sideC;

  float s = (sideA + sideB + sideC)/2;
  float area = sqrt(s*(s-sideA)*(s-sideB) * (s-sideC));

  cout << "\nThe area is: " << area;
  return 0;
}
```

EXAMPLE 1: TRIANGLES

C

```c
#include<stdio.h>
#include<math.h>

int main() {
  int sideA, sideB, sideC;
  printf("What is side A? ");
  scanf("%d", &sideA);
  printf("What is side B? ");
  scanf("%d", &sideB);
  printf("What is side C? ");
  scanf("%d", &sideC);
  float s = (sideA + sideB + sideC)/2;
  float area = sqrt(s*(s-sideA)*(s-sideB) * (s-sideC));
  printf("Area is: %f", area);
  return 0;
}
```

C++

```cpp
#include<iostream>
#include<cmath>
using namespace std;

int main() {
  int sideA, sideB, sideC;
  cout << "What is side A?";
  cin >> sideA;
  cout << "What is side B?";
  cin >> sideB;
  cout << "What is side C?";
  cin >> sideC;
  float s = (sideA + sideB + sideC)/2;
  float area = sqrt(s*(s-sideA)*(s-sideB) * (s-sideC));
  cout << "\nThe area is: " << area;
  return 0;
}
```

Main difference is what header files and functions they have access to.

EXAMPLE 1: TRIANGLES

C

```c
#include<stdio.h>
#include<math.h>

int main() {
  int sideA, sideB, sideC;
  printf("What is side A? ");
  scanf("%d", &sideA);
  printf("What is side B? ");
  scanf("%d", &sideB);
  printf("What is side C? ");
  scanf("%d", &sideC);
  float s = (sideA + sideB + sideC)/2;
  float area = sqrt(s*(s-sideA)*(s-sideB) * (s-sideC));
  printf("Area is: %f", area);
  return 0;
}
```

C++

```cpp
#include<iostream>
#include<cmath>
using namespace std;

int main() {
  int sideA, sideB, sideC;
  cout << "What is side A?";
  cin >> sideA;
  cout << "What is side B?";
  cin >> sideB;
  cout << "What is side C?";
  cin >> sideC;
  float s = (sideA + sideB + sideC)/2;
  float area = sqrt(s*(s-sideA)*(s-sideB) * (s-sideC));
  cout << "\nThe area is: " << area;
  return 0;
}
```

Both codes are almost identical.

PARTS OF C++ PROGRAM

```
#include<iostream>
#include<math.h>
using namespace std;

int main()
{
  int sideA, sideB, sideC;

  cout << "What is side A?";
  cin >> sideA;
  cout<< "What is side B?";
  cin >> sideB;
  cout<< "What is side C?";
  cin >> sideC;
  // Calculating semi perimeter
  float s = (sideA + sideB + sideC)/2;
  float area = sqrt(s*(s-sideA)*(s-sideB) * (s-sideC));

  cout << "\nThe area is: " << area;
  return 0;
}
```

Preprocessor directive

Namespace to use

Main function

Start of code block

PARTS OF C++ PROGRAM

```
#include<iostream>
#include<math.h>
using namespace std;

int main()
{
  int sideA, sideB, sideC;

  cout << "What is side A?";
  cin >> sideA;
  cout<< "What is side B?";
  cin >> sideB;
  cout<< "What is side C?";
  cin >> sideC;
  // Calculating semi perimeter
  float s = (sideA + sideB + sideC)/2;
  float area = sqrt(s*(s-sideA)*(s-sideB) * (s-sideC));

  cout << "\nThe area is: " << area;
  return 0;
}
```

Variable declaration

Input/Output

return statement

#INCLUDE DIRECTIVE

#WhatIsThis?

- This is the preprocessor directive
- Tells the preprocessor what files need to be included for the program to work.
- Doesn't need a ; at the end
- The content between <> is the name of the header file to look for.

```
#include<iostream>
#include<math.h>
```

NAMESPACE

- C++ uses namespaces to organize the names of program entities.
- `using namespace std;` indicates that we will use the entities in `std`.
  - Example: We can directly use: `cout << "Hello";`
- Otherwise, we must call `std` before whichever entity we want to use.
  - Example: `std::cout << "Hello";`

SPECIAL CHARACTERS

- Some character have special purposes or meanings, much like keywords.
- This can cause issues when trying to print them.

Character	Name	Description
//	Double Slash	Begins a single line comment
#	Pound Sign	Begins preprocessor directive
< >	Brackets	Encloses filename for preprocessor directive
( )	Parentheses	Used when naming functions and for conditionals
{ }	Braces	Encloses a group of statements
" "	Double Quote Marks	Encloses a string
' '	Single Quote Marks	Encloses a character
;	Semicolon	Marks the end of a statement

COUT

- `cout` **is used to output to the console.**
- **Example:** `cout << "Hello World";`
- `<<` **is used to send information**
  - **Can chain them to continue adding to the output.**
  - **Example:** `cout << "Hello World" << " I'm happy!";`

```
cout << "Hello World, " << "this is awesome!";
```



Hello World, this is awesome!

COUT

- `cout` **just concatenates strings, to move the cursor to the next line we need to add a newline.**
  - `cout << "Hello World" << endl;`
  - `cout << "Hello World\n";`
- **Both** `endl` **and including** `'\n'` **in the string have the same effect.**
- **The** `\` **before** `n` **indicates that this is an escape sequence, so the** `n` **should not be treated as a character. Instead, it indicates a new line.**

ESCAPE SEQUENCES

- These are used to change the position of a cursor.
- The also allow for special characters to be printed, like ', " and \

Escape Sequence	Name	Description
\n	Newline	Moves cursor to next line
\t	Horizontal tab	Moves cursor to next tab stop
\a	Alarm	Cause computer to beep
\b	Backspace	Moves cursor one position to the left
\r	Return	Moves to beginning of the current line
\\	Backslash	Allows for a backslash to be printed
\'	Single quote	Allows for a single quote to be printed
\"	Double quote	Allows for a double quote to be printed

CIN

- `cin` receives information from the console
  - Example `cin >> x;`
  - We are passing a value to `x`
  - `>>` is used to send the information
- Can receive multiple inputs at once, each input must be separated by a space or newline [Enter].
  - `cin >> x >> y >> z;`

CIN

- `getline(cin, str)` - can also be used to read input from the user.
  - `cin` - the source of the input, can be `cin` (console)
  - `str` – the variable we want to store the string in.
- Why use this instead of `cin`?
  - Can read input that includes whitespaces.

CIN

- To read a character we can use:
  - `cin >> ch;`
  - `cin.get(ch);`
  - `ch = cin.get();`
- To ignore input, we can use: `cin.ignore(c, n)`
  - `c` – if this character is found stop ignoring.
  - `n` – the number of characters to ignore.

VARIABLES AND ASSIGNMENTS

- Variables represent a storage location.
- It point to where in memory we are storing data.
- Two types of statements:
  - Declaration: No value is assigned.
  - Assignment: A value is assigned using =

```
int value; // Declaration
int value2 = 10; // Declaration and Assignment
```

VARIABLES AND ASSIGNMENTS

- A declared variable can be assigned later.
- Once declared, we no longer need to specify the type.
- Its value can be replaced, but its previous value cannot be recovered

```
int value; // Declaration
int value2 = 10; // Declaration and Assignment

value = 20; // Assignment
value2 = 12; // Replaced value
```

MORE ASSIGNMENTS AND AUTO

- For an assignment to be legal the left of the assignment must be the variable (LValue) and the right side the value (RValue).

```
int value1 = 10; // Valid
10 = value1; //Invalid
```

- Can initialize *several* values in one statement.

```
int value1 = 10, value2 = 15, value3;
 value1 = value2 = value3 = 25;
```

- A variable is initialized when we assign it a value.

```
int value = 10; // Initialized
int value2; // Declared but not initialized
```

MORE ASSIGNMENTS AND AUTO

- The `auto` keyword can be used instead of assigning a type.
- The compiler determines the type instead.
- Like variable assignments in Python.
- **Examples:**

```
auto initial = 'G'; // Will assign a char
auto value = 12; // Will assign an int
```

COMBINED ASSIGNMENTS

- Operators: +=, -=, *=, and /= are shorthand for both assigning and applying an operation.
- Operators: ++ and -- are shorthand for +=1 and -=1 respectively.

Operator	Equivalent
`var1 += 5`	`var1 = var1 + 5`
`var1 -= 5`	`var1 = var1 - 5`
`var1 *= 5`	`var1 = var1 * 5`
`var1 /= 5`	`var1 = var1 / 5`
`var1++`	`var1 = var1 + 1`
`var1--`	`var1 = var1 - 1`

VARIABLE NAMES

Legal identifiers:

- Cannot be a reserved word
- Can contain a mix of letters, numbers, and underscore (_).
- Can start with a letter or underscore, but not number.
- The case of the letters matters. `Value` is *not* the same as `value`.

Legal:

- `month`
- `employee_id`
- `_register1`
- `GraduationDate`

Illegal:

- `180Angle`
- `FoodColoring#12`

VARIABLE NAMES

Naming conventions:

- `camelCase`: Start in lowercase then each subsequent word starts in uppercase.
  - `birthDate`
  - `heightOfTheYoungestChild`
- `snake_case`: All lowercase and each word is separated by an underscore.
  - `birth_date`
  - `height_of_the_youngest_child`

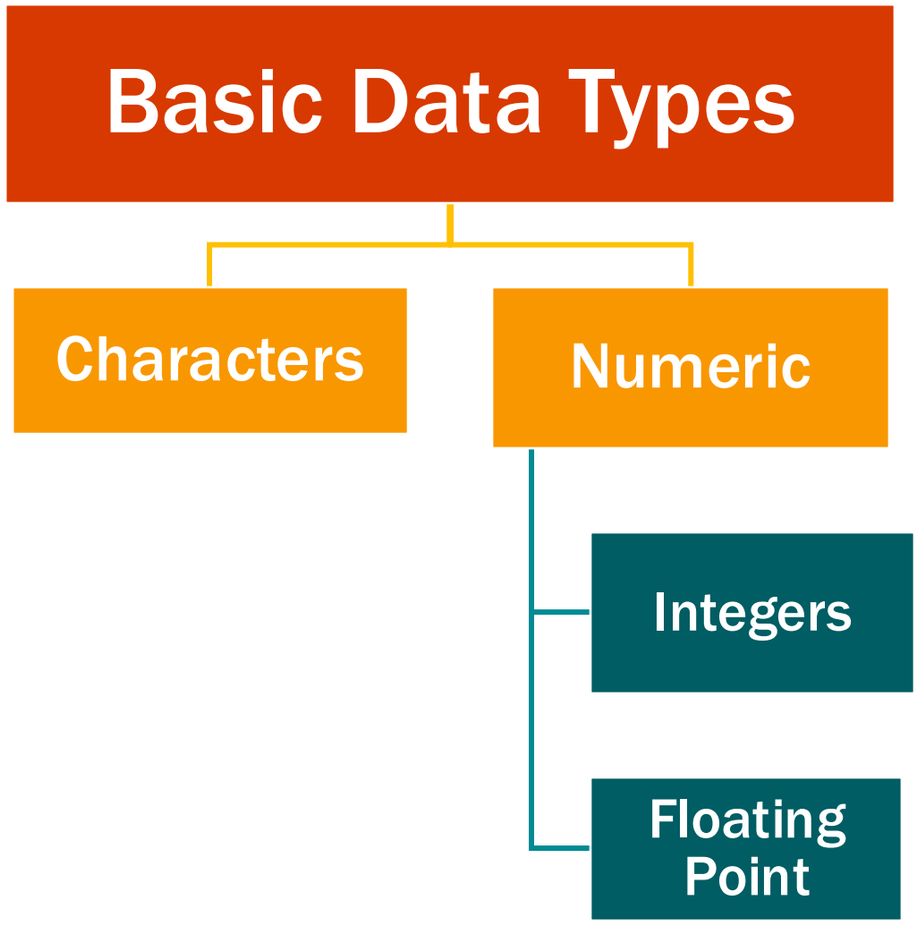
Good practice:

- Name must be indicative of the variables purpose.
- What does the variable represent?
- Prefer: `height = 10;`
- Avoid `h = 10;`

LITERALS

- Data that is written directly in the code.
- It explicitly states the value.
- Examples
  - `int integer_value = 15;`
  - `float weight = 152.12;`
  - `char alpha = 'a';`
  - `string hello = "Hello world";`

BASIC DATA TYPES



CHARACTERS

- This stores a single character.
- Simplest data type.
  - `char letterA = 'A';`
- Closely related to numbers.
- Often stored using ASCII encoding. Each character has an assigned numeric value.
  - The ASCII code for `'A'` is 65.

INTEGERS

- Numeric value with no fractional part.
- There are 4 types of integers we can use:
  - `short`
  - `int`
  - `long`
  - `long long`
- These can be signed or unsigned (only positive values).
- The difference is in how many values they can represent.

INTEGERS

Data Type	Usual size	Range
short int	2 bytes	-32,768 to 32,767
unsigned short int	2 bytes	0 to 65,535
int	4 bytes	-2,147,483,648 to 2,147,483,647
unsigned int	4 bytes	0 to 4,294,967,295
long int	4 bytes	-2,147,483,648 to 2,147,483,647
unsigned long int	4 bytes	0 to 4,294,967,295
long long int	8 bytes	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
unsigned long long int	8 bytes	0 to 8,446,744,073,709,551,615

In most cases we can omit the int, unless the datatype is int itself.
Examples:

```
short height;
unsigned long long weight;
```

INTEGERS

Why do the signed and unsigned represent such different values?

- It's all about the size.
- A short occupies 2 bytes.
  - 1 byte = 8 bits
  - 2 bytes = 16 bits
  - 16 bits can represent 2^{16} numbers (65,536 numbers).

INTEGERS

Why do the signed and unsigned represent such different values?

- When a number is signed, we are sacrificing the leftmost bit.
- The values that can be represented are reduced by half.
- For example, shorts are 16 bits so they can 65,536 numbers.
  - When unsigned the numbers go from 0 to 65,536.
  - When signed we can represent half that value (32,768), but it can be positive or negative.
  - Notice that we have one less positive value compared to negative, why is that?

`short int`	2 bytes	-32,768 to 32,767
`unsigned short int`	2 bytes	0 to 65,535

INTEGERS

Assume we want to represent numbers with only 4 bits.

- If it's unsigned that means we can represent numbers 0 (`0000`) to 15 (1111).
- Signed we can only represent from –8 to 7, since we only have access to the last three bit for the numbers.
- A number is positive if the leftmost bit is 0 and negative if it's 1.
  - `1000` (-8)
  - `0111` (7)

HEXADECIMAL AND OCTAL

- We can also represent binary, hexadecimal, and octal values.
- To represent binary, we use the prefix 0b. (zero-b)
  - `0b001` (1)
  - `0b101` (5)
  - `0b1101` (13)
- To represent hexadecimal, we need the prefix 0x. (zero-x)
  - `0x101` (5)
  - `0xF` (16)
- For octal we lead with a 0.
  - `010` (8)
  - `001` (1)

FLOATING POINT

- Represent numbers with a fractional parts.
- There are three types:
  - `Float`
  - `Double`
  - `Long double`
- The difference is in the level of accuracy.
- This is stored like scientific notation.

FLOATING POINT

Data Type	Usual size	Range	Significant digits
float	4 bytes	+/- 3.4E-38 to +/-3.4E38	7
double	8 bytes	+/- 1.7E-308 to +/-1.7E308	16
long double	8 bytes	+/- 1.7E-308 to +/-1.7E308	16

FLOATING POINT

- `Floats` are stored as `double` by default, to force it to be stored as a float we append `f` to the value.
  - `float value = 7.89f;`
- Can also represent floating point values with scientific notation
  - `double value = 1.3456E3;(1345.6)`
  - `float value2 = 2.278098E-4; (0.00027898)`
- If we want to force a value to be stored as a `long double`, we add `L` at the end.
  - `long double ld_value = 7.789L;`

MATHEMATICAL EXPRESSIONS

- Expressions are statements that evaluate to a value
- Can be assigned to variables
- There is an order of operations:

1. Expressions in parenthesis ( )
2. Negation from left to right
3. *, / and % in this order, from left to right
4. + and – in that order from left to right

MATHEMATICAL EXPRESSIONS

What effect does the order have?

Expression	Value
`1 + 5 * 3 - 4 / 2`	**14**
`(1 + 5) * 3 - 4 / 2`	**16**
`1 + 5 * (3 - 4) / 2`	**-1.5**
`(1 + 5) * (3 - 4) / 2`	**-3**

MATHEMATICAL EXPRESSIONS

Notice that:

- Multiplication *needs* to include an operator
  - (7)(8) //Doesn't work
  - 7 * 8 // Does work
- There is no exponential operator we must use a function.
  - x^2 => `pow(x, 2)`
  - x ** 2 doesn't work here

MATHEMATICAL EXPRESSIONS

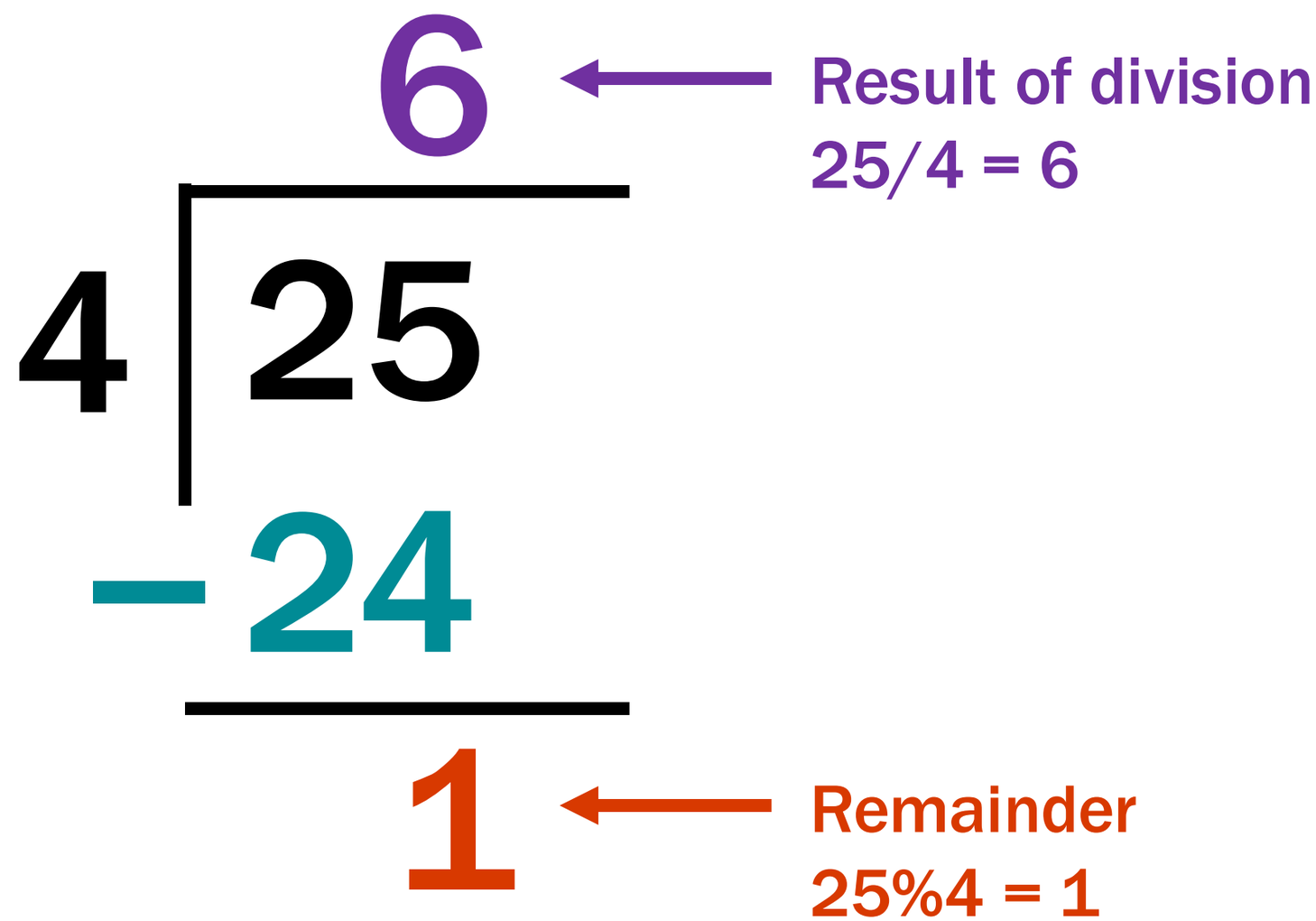
- Division between integers returns an *integer*, regardless of whether it has a decimal part.
  - `1/2  = 0`
- If either operand is a floating-point value, then the result will be a floating point.
  - `1 / 2.0 = 0.5`
  - `1.0 / 2 = 0.5`

MATHEMATICAL EXPRESSIONS

- The modulus (%) operator returns the remainder of a division.
- Both operators must be integers.
- Frequently used for finding if a value is *divisible* by a given number.
  - Check if value is even: `value % 2 == 0`
  - Check if divisible by 12: `value % 12 == 0`
- Great for restarting a counter.

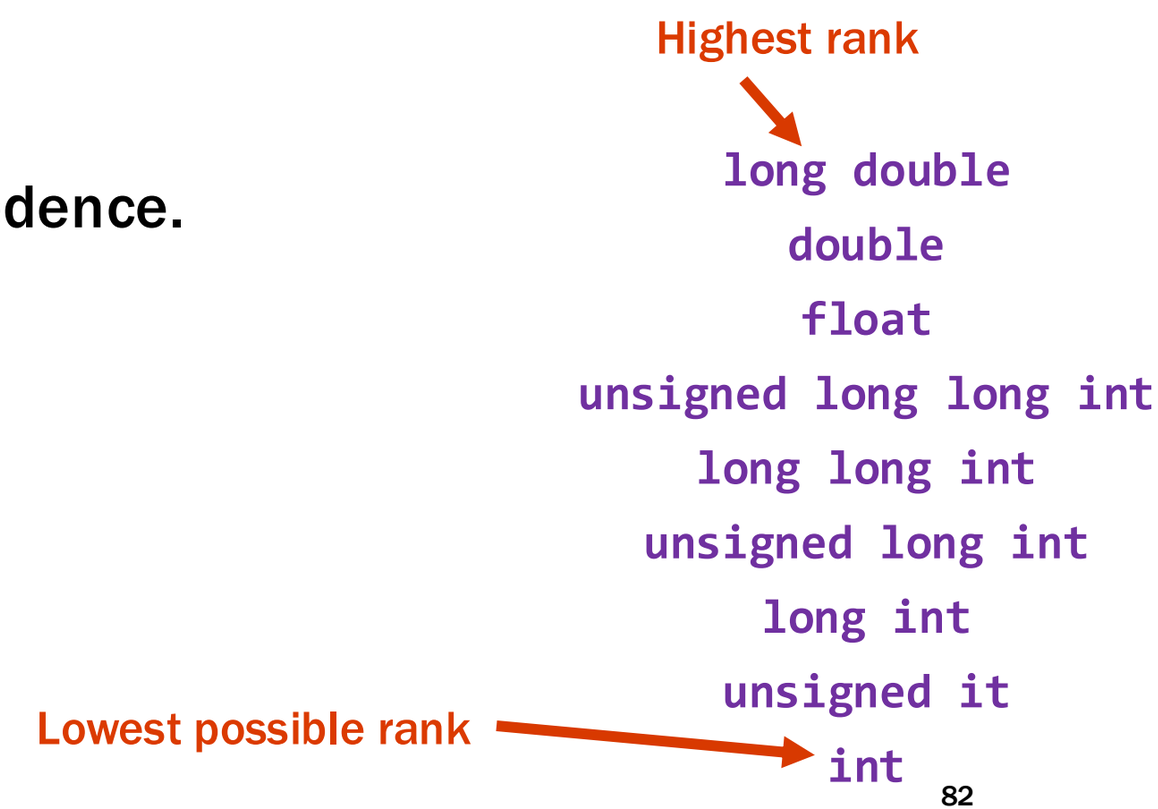
```
0 % 3 = 0    1 % 3 = 1    2 % 3 = 2
3 % 3 = 0    4 % 3 = 1    5 % 3 = 2
```

DIVISION VS MODULUS



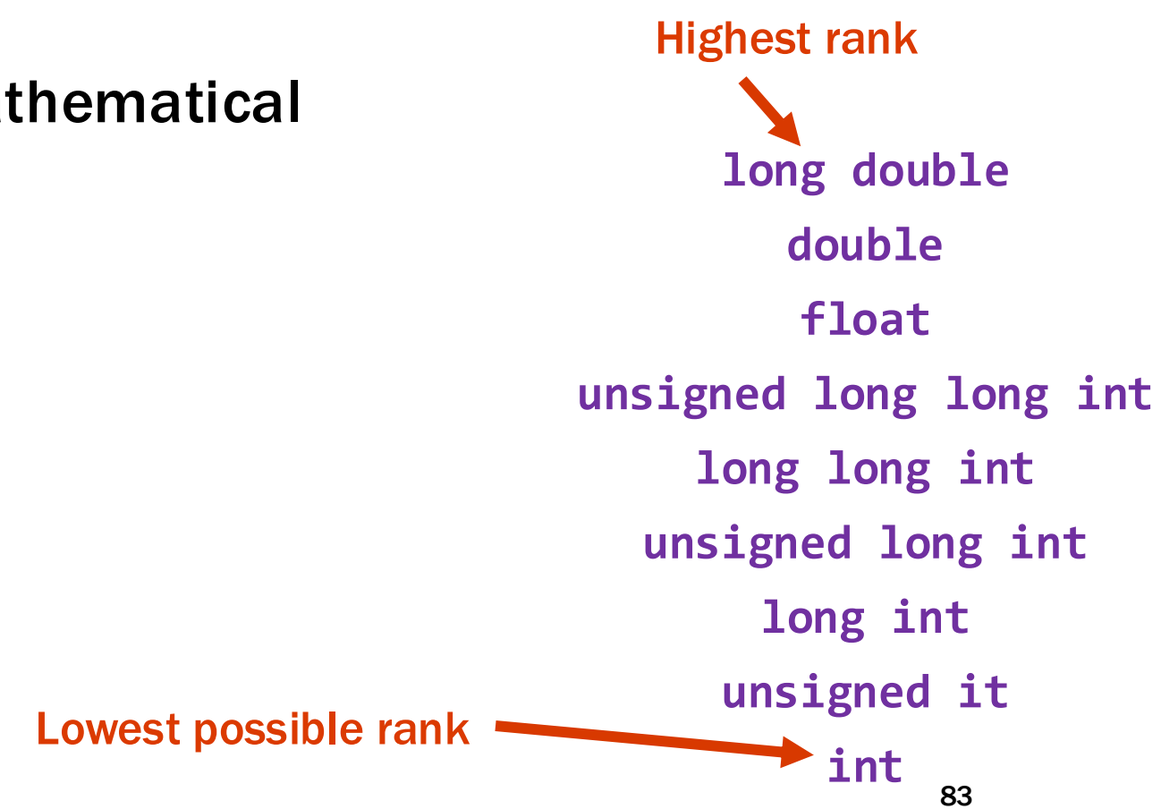
TYPE CONVERSION

- When you do operations between operands of *different* types C++ will automatically convert it.
- This is called coercion.
- This can influence the results.
- The more precise type takes precedence.
  - Highest rank is `long double`
  - The lowest possible rank is `int`



TYPE CONVERSION

- `Chars` and `shorts` get promoted to `int`
- When assigning (=) the type on the right gets converted to the type on the left.
- We can mix numerical types in mathematical expressions.



TYPE CONVERSION

Definitions:

- Coercing: Automatic conversion
- Promotion: Converts value to higher rank
- Demotion: Converts value to lower rank. This can result in loss of precision and even incorrect values.

```
int value = 'a';       Promotion char -> int (value = 65)
double value2 = value; Promotion int -> double (value2 = 65.00000000)
float value3 = 10.56f + value; Promotion int -> float (value3 = 75.56000)
int value4 = value3; Demotion float -> int (value4 = 75)
```

CASTING

This is when we convert a variable of one type to another manually.

- **Standard way**: `static_cast<type>(variable)`

  ```
  age = static_cast<int>(number/12)
  ```

- **C-style**: `(type) variable`

  ```
  age = (int) number/12
  ```

- **Pre-standard**: `type (variable)`

  ```
  age = int (number/12)
  ```

OVERFLOW/UNDERFLOW

- When we assign a value that is too big or too small.
- Can occur with either integer or floating points.
- When it occurs, the value becomes gibberish.
- Example:
  - If we only have space for 4 digits, we can store numbers from 0 to 9999.
  - What happens if we add 1 to 9999?
  - The result is 10000, but there's only room for 4 digits. The extra digit is lost, and the stored value becomes 0000.
  - This means the calculation is correct, but the stored value is wrong because it exceeds the available space, **the number *overflows*.**

OVERFLOW/UNDERFLOW

- Overflow is handled differently by system:
  - Can display a warning
  - Stop the program
  - Continue execution

CONSTANTS

- Defined when assigned
- Cannot be changed later
- Name usually in all upper case
- Uses the `const` keyword
- Examples:
  - `const PI = 3.14`
  - `const MAXIMUM_CAPACITY = 32`

STRING CLASS

- Holds a sequence of characters.
- Not in C++ standard library, must include `#include<string>`
- Stored in consecutive memory.
- A null value is appended to know where the string ends in memory.

S	t	r	i	n	g	\0

STRING OPERATORS

Useful string functions:

- `length()`: returns how many characters the string has excluding the null character.
- `assign(n, c)`: puts a repeated character in a string.
  - n – times to repeat
  - c – character that will be repeated

STRING OPERATORS

Useful string functions:

- `at(i)`: returns character at position `i`.
- `[i]` operators achieve the same effect
  - Example: `word = "Hello";`
  - `word.at(1)` returns `e`
  - `word[1]` returns `e`
  - Difference is that `at()` does bound check.
- `to_string(x)`: Converts x into a string.

STRING OPERATIONS

Concatenation:

- Strings can be joined together with the + symbol
- The += works as well.

```
string str1 = "Hello";
string str2 = "World";
string str3 = str1 + " " + str2; // Hello World
```

BOOLEAN

- Can only be two values: true or false.
- Also stored as integers 1 is true and 0 is false.
- You can assign a conditional
  - `bool condition = 12 < 18;`
- When printing a boolean, by default, it prints 0 or 1.
- Can use `cout << boolalpha;` to get a true or false instead.

SCOPE

- Determines where a variable can be used.
- Also determines end of life. When memory for that variable is released.
- If a two variable share a name, it determines which of the two precedes.
- Local variable takes precedence.
- A variable can't be used *before* it's defines.

COMMENTS

- Comments are lines of code that are ignored by the compiler
- Useful for the programmer to leave notes about the code
- Single line comments are preceded by `//`
  - `int value = 10; // This will be ignored`
- Multi line comments are enclosed in `/* */`
  - ```
    /* This comment has
           More than one line
           And it end here */
    ```
    ```

C++ STANDARD LIBRARIES

C++ STANDARD LIBRARIES

- These are collections of of classes and functions that are included with C++.
- The provide access to useful and common features that are frequently needed.
- This includes data structures, algorithms and memory management.
- Some frequently used libraries:
  - `iostream`
  - `string`
  - `cmath`
  - `vector`

MATH FUNCTIONS

- Common math functions can be accessed using the cmath header file.
- The functions receive and return doubles.

Function	Description
`abs`	Absolute value
`sin`	Sine (It receives an angle in radians)
`cos`	Cosine (It receives an angle in radians)
`tan`	Tangent (It receives an angle in radians)
`sqrt`	Square root
`log`	Natural logarithm
`pow`	Raise to the power

CHAR FUNCTIONS

- Common char functions can be accessed using the cctype header file.
- The functions receive characters and return a boolean.

Function	Description
`isalnum`	Checks if the character is alphanumeric
`isalpha`	Checks if the character is alphabetic
`isblank`	Checks if the character is blank
`isdigit`	Checks if the character is a digit
`islower`	Checks if the character is lower case
`ispunct`	Checks if the character is a punctuation character
`isxdigit`	Checks if the character is a hexadecimal digit

STRING FUNCTIONS

- The don't receive a string, they are accessed through the string variable.
- They can receive other info as parameters.
- Some common functions are:

Function	Description
`at`	Access character at that position (starts at 0)
`empty`	Checks if the string is empty.
`append`	Adds character at the end.
`insert`	Adds a character.
`clear`	Empties the string. (no content)
`replace`	Replaces a portion of the string.
`substr`	Returns a substring.

RANDOM NUMBERS

- To generate random numbers, we need the `cstdlib` header file.
- `rand()`: generates a random number between 0 and largest `int` the system can generate
- Every time the program runs it will generate the same sequence of values, unless we change the seed.

```
srand(15);
// Printing 5 random values
cout << "Printing 5 random values\n";
for(int i = 0; i < 5; i++)
    cout << rand()<<endl;
```

Output first run:

```
10254
12
1598
322
```

Output second run:

```
10254
12
1598
322
```

RANDOM NUMBERS

- `srand(x)`: It sets the seed value. This determines the sequence we will get.
- Should be called only once in the program.
- If we change the seed the sequence produced by `rand()` changes.

```
srand(x);
// Printing 5 random values
cout << "Printing 5 random values\n";
for(int i = 0; i < 5; i++)
    cout << rand()<<endl;
```

Output when x = 15:

```
10254
12
1598
322
```

Output when x = 3:

```
756
4789
12369
8
```

RANDOM NUMBERS

- To control the range of the random number we can use:

```
rand() % (max + 1)
```

- Where max is the upper limit, we want.
- For example, if we wanted a random number between 0 and 6, we would use:

```
rand() % (6 + 1)
```

```
int max = 10;
cout << "Print 5 random values between 1 and " << max << endl;
for(int i = 0; i < 5; i++)
    // Modulus forces the number to be between 0 and max - 1
    // Plus 1 to shift it to 1 and max
    cout << (rand() % max) + 1 <<endl;
```

Output:

```
0
5
7
5
```

CONCLUSION

- Introduced the concept of programming and why it's relevent.
- Introduced C++.